

SET2 A4. Значительные инверсии

1. ДаС-алгоритм CINV

Псевдокод:

CINV(A):

 n = len(A)

 if n <= 1:

 return (A, 0)

 mid = n/2

 left = A[0:mid]

 right = A[mid:n]

 (left_sort, left_inv) = CINV(left)

 (right_sort, right_inv) = CINV(right)

 (merged_sort, split_inv) = COMBINE_PARTS(left_sort, right_sort)

 total_inv = left_inv + right_inv + split_inv

 return (merged_sort, total_inv)

COMBINE_PARTS(L, R):

 merged = []

 i = 0, j = 0

 split_inv = 0

 len_L = len(L), len_R = len(R)

 while (i < len_L and j < len_R):

 if L[i] <= R[j]:

 merged.append(L[i])

 i += 1

 else:

 merged.append(R[j])

 split_inv += (len_L - i)

 j += 1

 merged.append(L[i:])

 merged.append(R[j:])

 return (merged, split_inv)

Пояснение:

DIVIDE: Разбиваем массив A на две примерно равные половины: left и right

CONQUER: Рекурсивно вызвать CINV для обеих половин. Получим отсортированные половины и количество инверсий внутри каждой.

COMBINE: Сливать половины в один отсортированный массив, параллельно подсчитывая инверсии. При сравнении элементов L[i] и R[j] из левой и правой половин, если L[i] > R[j], то

все оставшиеся элементы в левой половине (начиная с i) тоже больше $R[j]$. Значит, они образуют инверсии с $R[j]$. Количество инверсий увеличивается на $(\text{len}(L) - i)$.

Рекуррентное соотношение:

Алгоритм рекурсивно обрабатывает две половины массива (по $n/2$ элементов каждая), а затем выполняет слияние с подсчётом инверсий за линейное время $O(n)$. Это даёт следующее уравнение: $T(n) = 2T(n/2) + O(n)$

По мастер-теореме: $a = 2$, $b = 2$, $f(n) = O(n)$, $n^{\log_b a} = n^{\log_2 2} = n$ - сравним с $f(n)$, т.к. $k = \log_b a$ (случай 2) $\Rightarrow T(n) = O(n^k \cdot \log n) = O(n \cdot \log n)$ ч.т. д.

Возвращает ли алгоритм минимальное количество инверсий?:

Да, алгоритм CINV возвращает минимальное количество необходимых инверсий, т.к. инверсия - пара элементов $(i < j, a_i > a_j)$ требует одной перестановки, и алгоритм подсчитывает все такие пары в исходном массиве. Поскольку в модели данной задачи каждая пара требует ровно одной перестановки для устранения, полученное число и будет являться минимальным количеством необходимых инверсий.

2. Изменение в алгоритм CINV

На этапе слияния двух отсортированных половин нужно добавить дополнительный проход для подсчёта значительных инверсий.

Псевдокод:

CINV(A):

$n = \text{len}(A)$

 if $n \leq 1$:

 return (A, 0)

$\text{mid} = n / 2$

$\text{left} = A[0:\text{mid}]$

$\text{right} = A[\text{mid}:n]$

$(\text{left_sort}, \text{left_sig_inv}) = \text{CINV}(\text{left})$

$(\text{right_sort}, \text{right_sig_inv}) = \text{CINV}(\text{right})$

$(\text{merged_sort}, \text{split_sig_inv}) = \text{COMBINE_PARTS}(\text{left_sort}, \text{right_sort})$

$\text{total_sig_inv} = \text{left_sig_inv} + \text{right_sig_inv} + \text{split_sig_inv}$

 return (merged_sort, total_sig_inv)

COMBINE_PARTS(L, R):

 merged = []

$i = 0, j = 0$

$\text{len_L} = \text{len}(L), \text{len_R} = \text{len}(R)$

 split_sig_inv = 0

$i = 0$

 for $j = 0$ to $\text{len_R}-1$:

 while $p < \text{len_L}$ and $L[i] \leq 2 * R[j]$:

$i += 1$

```
split_sig_inv += (len_L - i)

while (i < len_L and j < len_R):
    if L[i] <= R[j]:
        merged.append(L[i])
        i += 1
    else:
        merged.append(R[j])
        j += 1

merged.append(L[i:])
merged.append(R[j:])
```

Асимптотическая сложность второго алгоритма: $O(n \log n)$, потому что на каждом уровне рекурсии мы выполняем два последовательных линейных прохода $O(n) + O(n) = O(n)$, а глубина рекурсии составляет $O(\log n)$. По мастер-теореме для $T(n) = 2T(n/2) + O(n)$ получаем $O(n \log n)$ (п1).